Unit - IV

Java Server Pages: JSP: Introduction, Architecture of JSP, Life Cycle of JSP, scripting elements (scriplets, JSP Declarations, JSP Expression), directive Elements (Page, include, taglib), Jsp Actions (include, setproperty, getproperty, forward, text), Implicit objects (request, response, out, page, exception, including HTML in JSP.
Introduction to JDBC.

Java is a popular computer programming language used to create software, websites, mobile apps, and games. It was developed by SUN microsystem in 1995 and is now owned by Oracle Corporation

Basic concepts of OOP's

It is a programming method that uses objects and classes to create programs. It helps make programs easy to understand, reuse, and maintains

1) Class

A class is a blueprint used to create object. It defines the data (variable) and methods (functions), that an object will have.
or
A class is a blueprint / template for creating objects and it defines properties (data) and behaviours (methods) that objects of that class can be created

- A class is a user defined data type
- It contains variables and methods
- multiple objects can be created from a single class

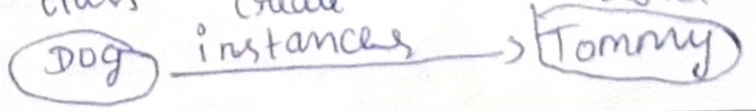
Ex:- Bank account is a class it contains data and methods

data :- name, account number, Balance
methods :- deposit(), display Balance()

2) Objects

An object is an instance of a class that represents a real word entity. It is used to access the variable and methods defined inside a class

Ex:- Dog is a class, tommy is an object of a class



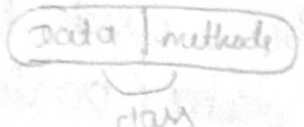
property / variable	methods
Name: Tommy	get name()
colour: white	get colour()

3) Data Abstraction

It is the process of hiding implementation details and showing only the essential features of an object.

Ex:- An ATM or a coffee machine represents abstraction where the user interacts with simple operations while the internal working and implementation details remain Hidden.

4) Encapsulation.

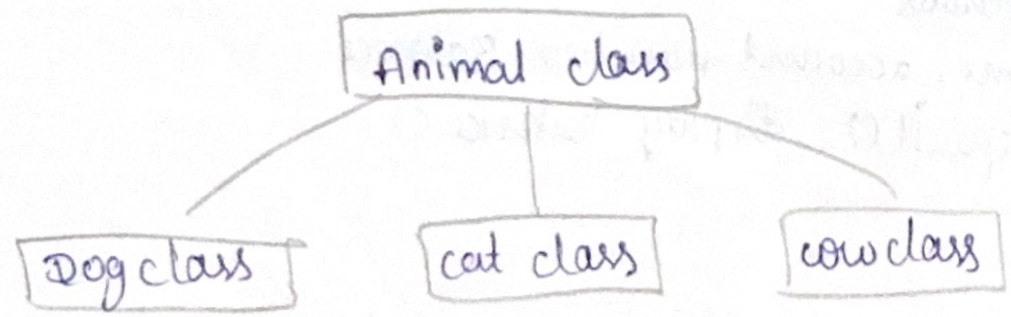


It is the process of wrapping data and methods into a single unit is called encapsulation

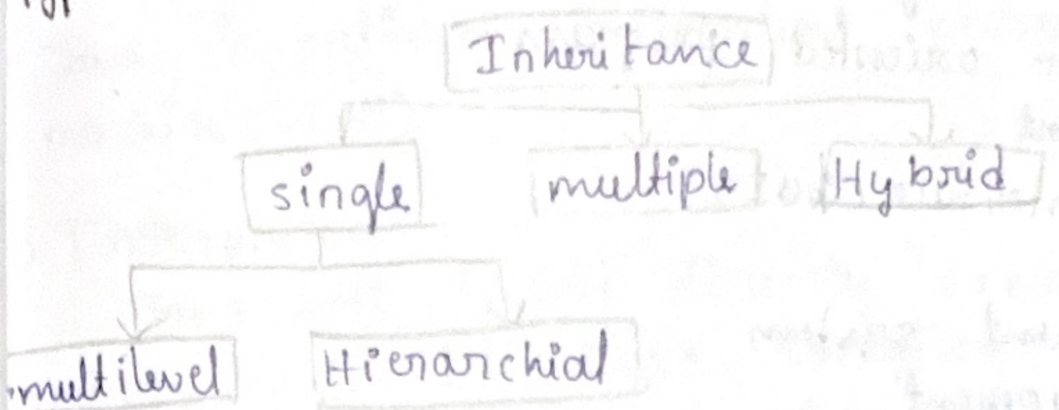
Inheritance

Inheritance is poor concept in Java that allows one class two acquire the fields and methods of another class using the extends keyword.

ex:- Dog, cat, cow, canbe child class of animal parent class



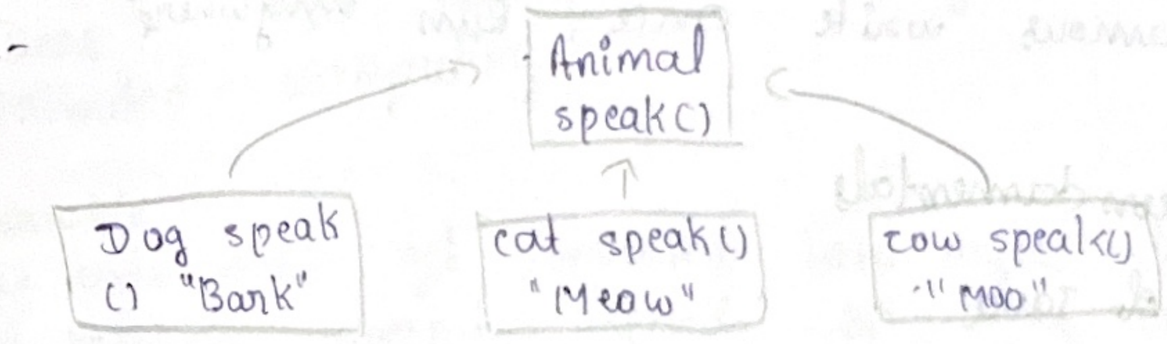
Type of inheritance



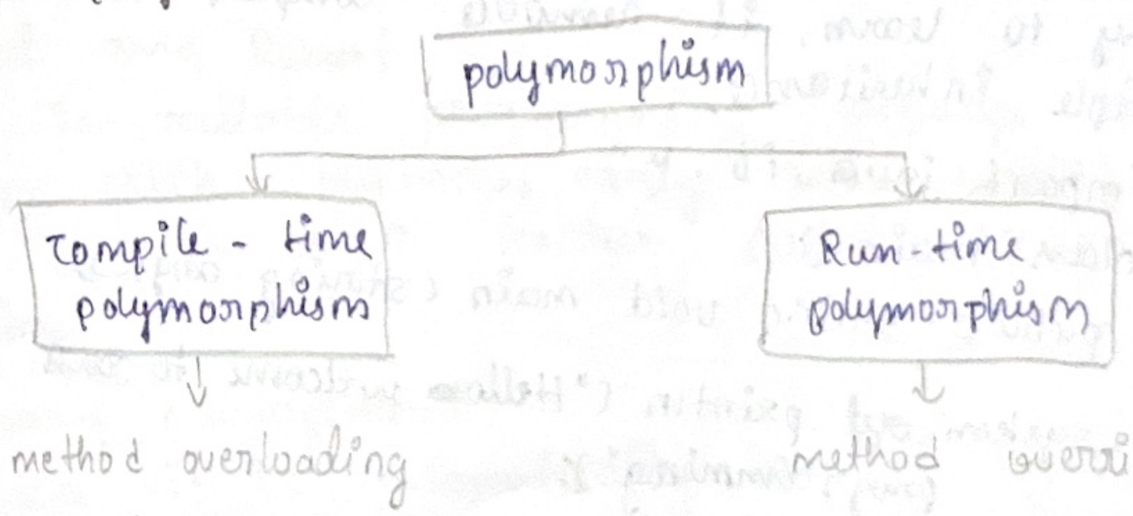
5. polymorphism

polymorphism means "one name many forms" where a single entity can behave differently in different situations In Java it allows the same method or object to show different behaviours based on the context

Ex:-



Type of polymorphism:



Imp

» Benefits of OOP's

-> code Reusability.
-> simple trouble shooting
-> prevents data redundancy
-> modularity
-> code maintenance
-> code security
-> scalability and extendability

Applications of object oriented program
-> software development
-> graphical user interface (GUI)
-> game development
-> database management system
-> mobile app development

Java virtual machine (JVM)
-> Java virtual machine is a core component of Java runtime environment (JRE) that allows Java programs to run on any platform without modifications.
-> JVM acts as an interpreter b/w Java bytecode and the computer hardware and it providing Java's famous write once, Run anywhere (WORA)

Java fundamentals

Features of Java

1) simple syntax - It is very straight forward and very easy to learn, it removes complex features and multiple inheritance.

ex: import java.io.*;
class main {
public static void main (string args[])
{ system.out.println ("Hello welcome to Java programming");
}

2) object oriented - Java is a pure object oriented language it supports core oop's concepts like class, objects, data abstraction, encapsulation, inheritance, and polymorphism.

3) platform independent
Java is platform independent because of Java virtual machine (JVM)

4) Interpreted
Java code is not directly executed by the computer, it is first compiled into bytecode, this bytecode is then understand by the (JVM) and enables Java to run on any plat form without rewriting the code

5) Scalable
Java can handle both small and large scale applications because Java provides features like multiple threading and distributed computing

6) portable
the code first get converted into byte code and this bytecode doesn't depends on any operating system or any specific computer.

7) secured and Robust
Java is reliable programming language because it can catch mistakes early while writing the code, it also as a feature called exception handling.

8) memory management
In Java it is automatically handled by Java virtual machine

9) High performance
Java is faster than old interpreted languages

10) multithreading
It allows multiple threads to run at the same time

Overview of Java

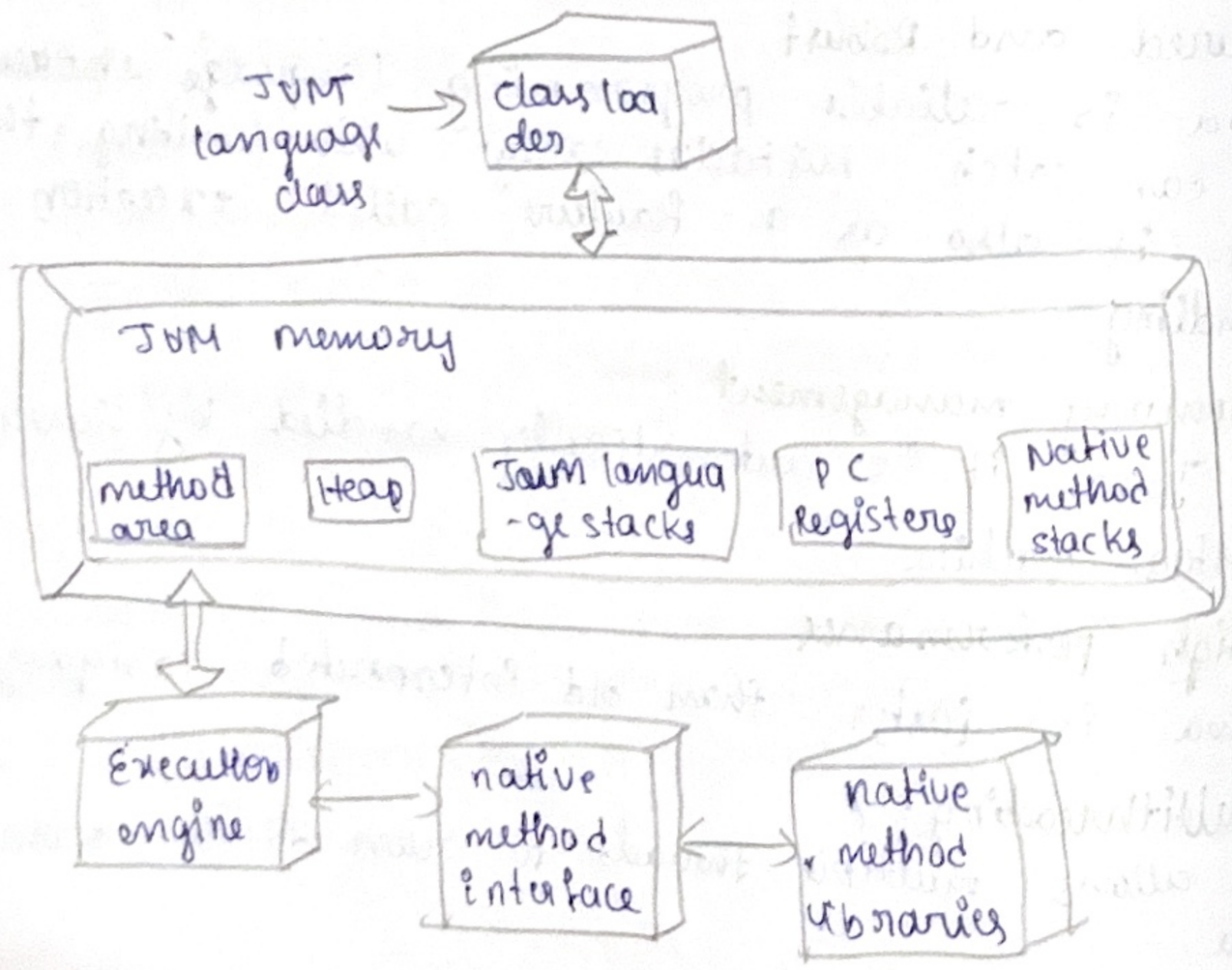
-> Java programming language was originally developed by sun microsystems. which was introduced by James gosling and released in the year 1995 as a core component of sun microsystems Java platform (Java 1.0)

-> James gosling initiated Java language project in June 1991. The language initially called oak. after an oak tree that's stood outside gosling's office, also went by the name green and ended up later being renamed as Java from a list of random words.

-> sun released the first public implementation as Java 1.0 .in 1995 it promised write once run anywhere (WORA)

oak tree is a national tree of USA & russia.

Architecture of JVM



09/7/2026

Constants

constants in Java are the fixed values those are not changed during the execution of program.

-> Java supports several type of constants they are.
* integer constants
* real constants
* single character constants
* string constants.
* Backslash constants. (\n, \t)

In Java we can create constant using the final keyword

Ex:- syntax:-

final datatype ~~final~~ constant_name = constant_value;

ex:- final double Pi = 3.142;

Example program

```
Public class constant example
{
public static void main (string args[])
{
final double PI = 3.142;
system.out.println ("PI");
}
}
```

Variable.

A variable is a named memory location used to store a value that can change during program execution.

syntax. datatype variable_name = variable value;

ex: int age = 21;

```
public class varible example
{ public static void main (String args[])
{ int age = 21;
System.out.println ("age");
} }
```

rules for declaring a variable.

1. A variable must be declared with a datatype.
2. Variable name must beings with a letter or (_) underscore and cannot start with a number
3. after the first character, numbers are allowed.
4. White space are not allowed.
5. Java key words cann't be used as variable names
6. Java variable names are case sensitive.

Data types.

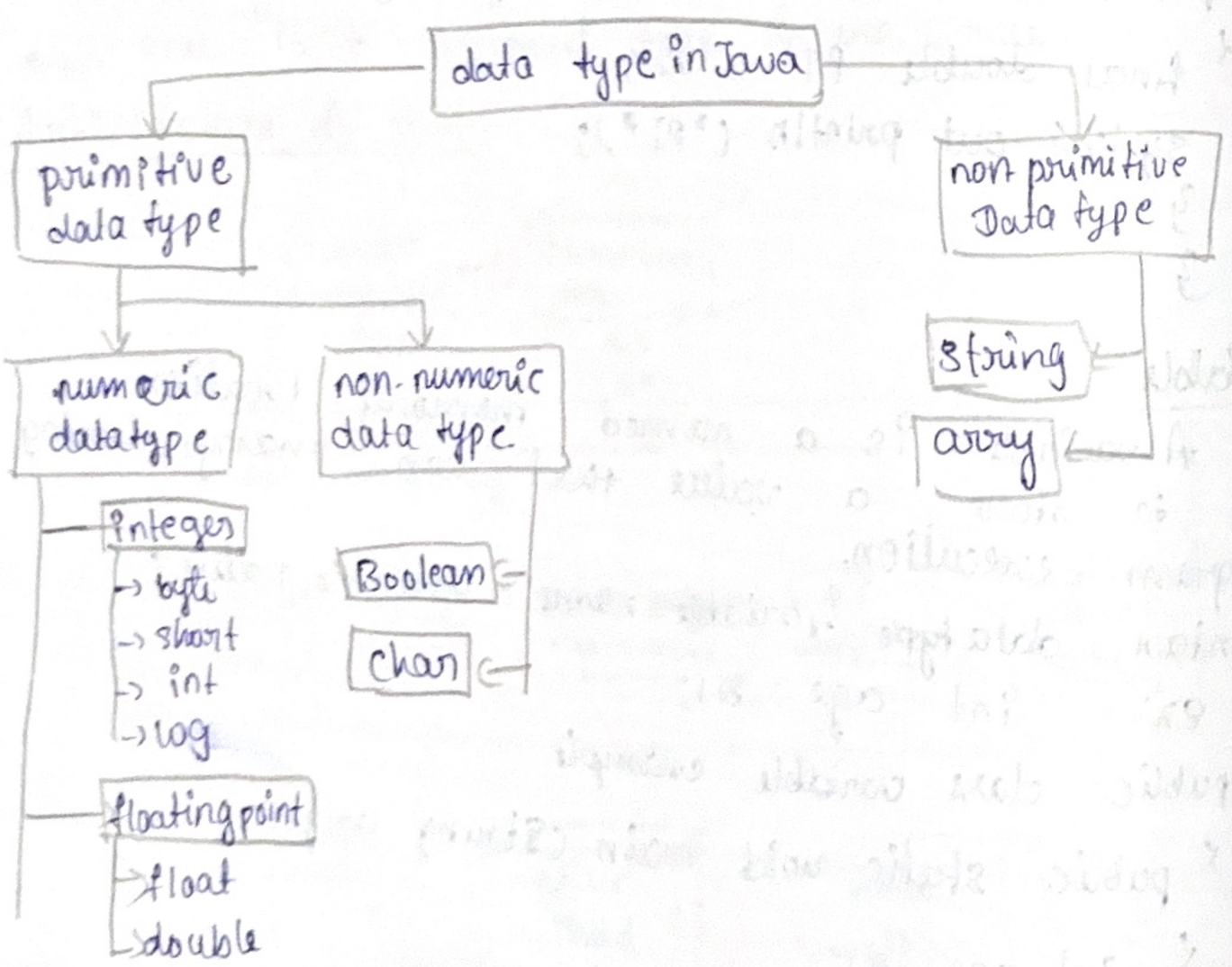
Datatypes defines the type of data a veriable can store in a program.

-> Java provides 2 mean types: 1) primitive
2) non-primitive

syntax:

datatype variable_name = value;

Types of Data type.



Operators

operators are the ~~simples~~ ~~use~~ symbol used to perform operation on variables and values.

-> They plays a key role in expressions, calculations and ~~dis~~ decision making in programs.

Ex:- 5+6

where, + is the operator and 5 & 6 are the operands

Types of operators

Name	operators
Arithmetic operators	+, -, *, /, %
logical operators	&&, ||, !
unary operators	--, ++
Ternary operations ~~ors~~	cond? true : false; (? :)
Assignment operators	+=, -=, *=, /=, %=, =
Bitwise & shift operators	&, |, ^, <<, >>, >>>
Relational operators	>, <, >=, <=, !=, ==
Instance of operator	obj instance of integer

Expresions in Java.

An expresion is combination of operators, constants and veriables.

-> An expresions may consist of one or more operands and 0 or more operators to produce a value.

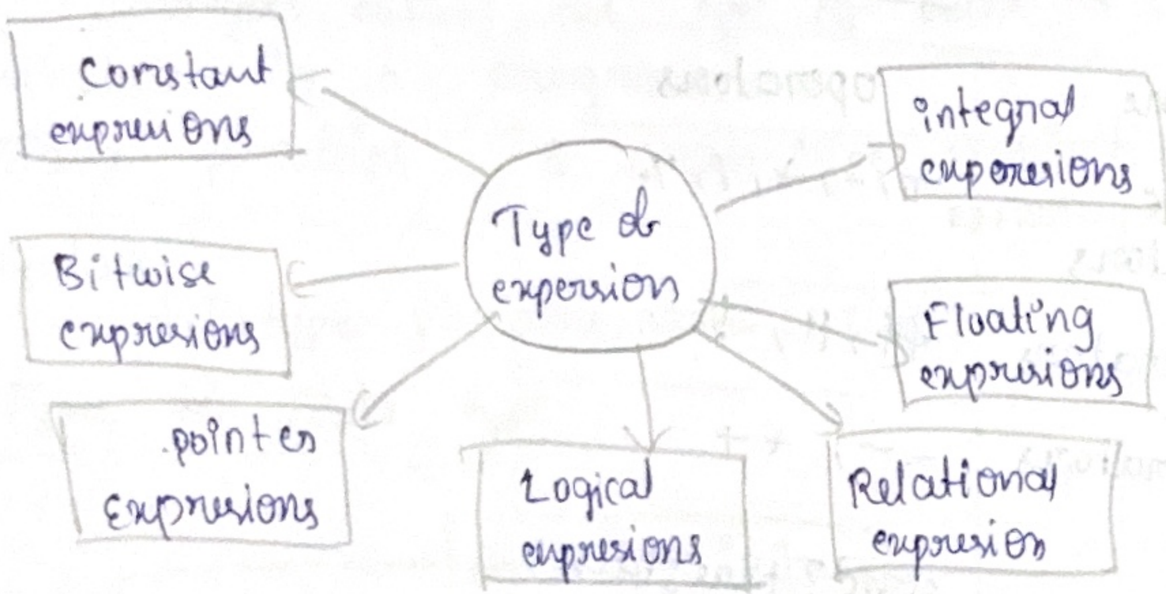
Ex:- result = a + b * c . . .

result → variable to store the expression value

a → operand1, + → operator1, b → operand2, * → operator2, c → operand3

Ex: a+b , c , s-1(7)*f . . . etc.

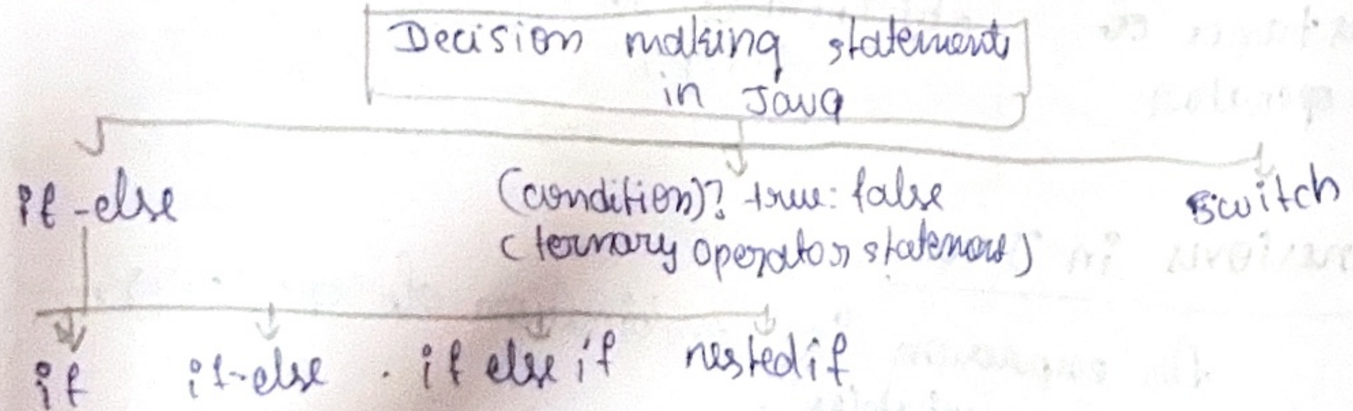
Type of expressions.



Decision making and Branching

Decision making and Branching statement in Java allows a program to brach sequential execution and to choose different execution paths based on specified conditions.

Types of decision making statements



→ Java if statement

→ The if-statement is the simplest decision making statements. It executes the block of code, only if the given condition is true

syntax.

```
if (condition)
{
  //true
}
```

Ex:-

```
public class Main
{
public static void main (String args[])
{
  int i=10;
  if (i<15)
  { System.out.println("The condition is true");
  }
}
}
```

→ Java if-else statement

→ The if else statement allows due to execute one block if the condition is true and another block if it is false.

syntax

```
if (condition)
{
  // true
}
else
  { // false
  }
```

Ex:-

```
import java.util.*;
public class main
{
public static void main (string args[])
{ int i=10;
  if (i<15)
{ System.out.println("i is smaller than 15");
}else{ system.out.println("i is grath than 15");
```

→ Java if-else-if statement. (if-else-if-ladder)
→ The if-else-if statement is used when we need to check multiple conditions one after another.

Syntax:

```
if (condition1)
  { //true
  }
else if (condition2)
  { //true
  }
else
  { //false
  }
```

Ex:-

```
import.java.util.*;
public class main
{ public static void main (String args[])
  { int marks = 75
    if (marks >=90)
     { system.out.println ("Grade A+");
     }
    else if (marks >=75)
     { system.out.println ("Grade A");
     }
    else if (marks >=60)
     { System.out.println ("Grade B");
     }
    else if (marks >=50)
     { system.out.println ("Grade C");
     }
    else
     { system.out.println ("Fail");
     }
  }
}
```

Java nested if statement.

→ A nested if statement mean an if statement written inside another if statement. It is used when a second condition should be checked only if first condition is true.

syntax:

```
if (condition1)
  { if (condition2)
    { //statements
    }
  }
```

Ex:-

```
import java.util.*;
public class Eligibility
{ public static void main (string args[])
  { int age = 20;
    boolean citizen = true;
    if (age >=18)
     { if (citizen = true)
       { system.out.println ("Eligible to vote");
       }
     }
  }
}
```

- Switch statement:- switch statement is a multiway branching statement. It provides an easy way to dispatch the execution to different parts of the code based on the value and expression

syntax.

```
switch (expression);
{case 1 : //statement 1;
 break;
 case 2 : //statement 2;
 break;
 .
 .
 case n : //statement n;
 break;
 default; //statement;
}
```

Ex:- import java.util.*;

```
class switch example {
public static void main (string args[])
{
   int num=20;
   switch (num)
{
case 1: system.out.println("It is 10");
 break;
case 2: system.out.println("It is 20");
 break;
default: system.out.println("Not present");
 }
 }
 }
```

→ Ternary operator statement. It is an alternative of if-else statement that evaluates the test condition and executes a block of code based on the result of the condition

syntax:

condition ? expression1 : expression2;

Here, condition is evaluated

↓

if condition is true, expression1 is executed

↓

if condition is false, expression2 is executed

The ternary operator takes three operands

→ condition, expression, and expressions Hence the name ternary operator

Ex class ternary operator

```
{
public static void main (String args[])
{ int a=10, b=20;
  int result= (a>b)?a:b;
  system.out.println(result);
}
```

looping statements in Java.

looping statements are used to execute a block of code repeatedly until a specified condition is met.

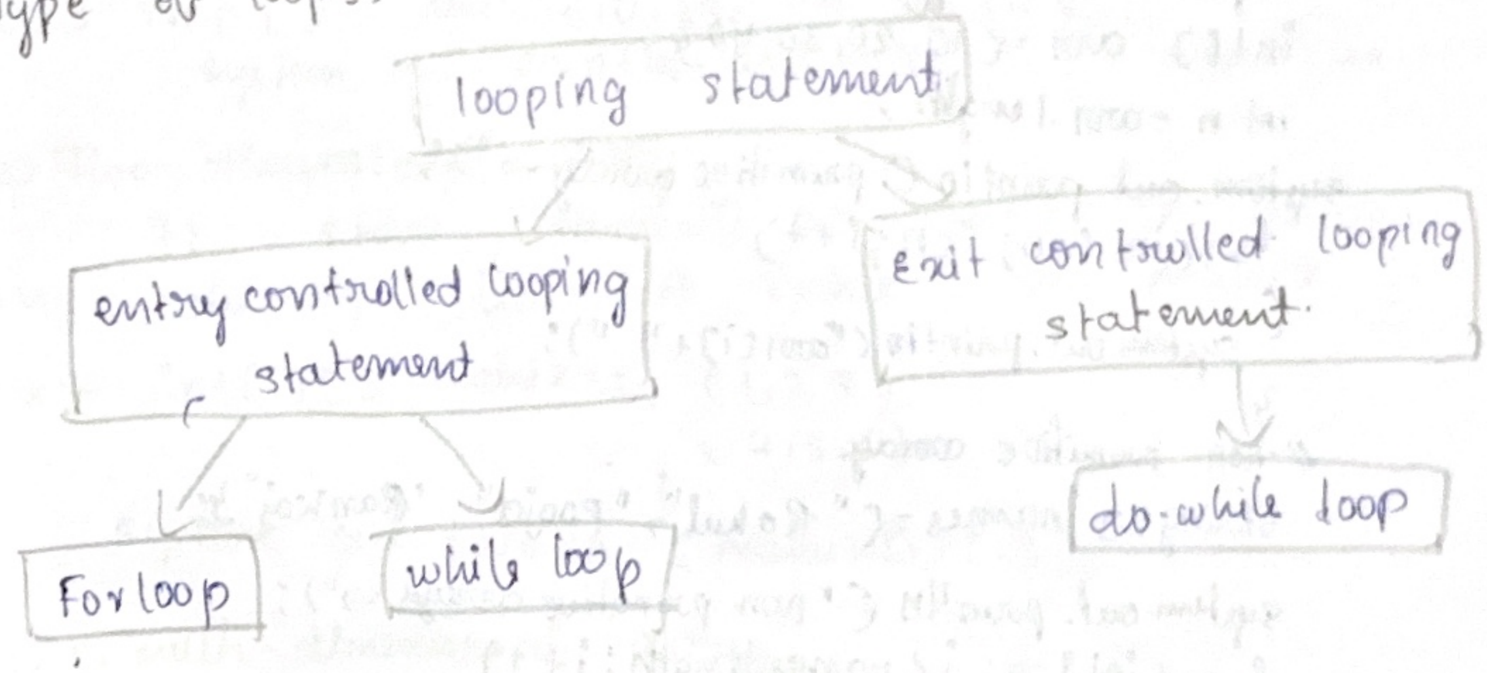
advantages

→ It reduce the code duplication

→ It allows execution of code multiple times

→ It improves efficiency by avoiding repeatitive code

Type of loops:-



syntax for For

```
for(initialization ; condition; updation)
{
// statements
}
```

syntax for while loop

```
while (condition)
{// statements;
 }
```

syntax for do-while loop

```
do {
// statements
}
while (condition);
```

Arrays

An array is a ordered collection of element of the same data type stored in contiguous memory locations.

→ It allows multiple values to be stored under a single name and accessed using an index.

Ex:-
```
public class Array {
    public static void main (string args[])
    { //primitive array
      int[] arr = {10, 20, 30, 40};
      int n = arr.length;
    system.out.println ("primitive array →");
      for (int i=0; i<n; i++)
      {
        system.out.println (arr[i]+" ");
      }
    // non- primitive array
      string[] names = {"Rahul", "Pooja", "Pankaj"};
      system.out.println ("non primitive arrays →");
      for (int i=0; i<names.length; i++)
      {
        system.out.println (name[i]+" ");
      }
    }
}
```

output: primitive array → 10 20 30 40
non-primitive array → Rahul pooja pankaj

Declaring an Array

syntax:
- data type[] array name;
- data type array name[];

Initialization of array

syntax:- int arr[] = new int[size];

Ex:- int arr[] = {1, 2, 3, 4, 5};

int arr[] = new int[5]

[0] [0] [0] [0] [0]

int arr[] = {10, 20, 30, 40, 50};

[10] [20] [30] [40] [50]

Types of arrays.

1) one-dimensional array

It store elements in a single row.

Ex:- int[] arr = {10, 20, 40, 30, 50};
system.out.println(arr[2]); output: 40

2) Two-dimensional array

It store elements or data in rows and columns like A table or matrix.

Ex:-
```
int[][] matrix = { {1,2,3}
                   {4,5,6}
                 };
system.out.println (matrix[1][2]);
```

3) multi-dimensional array

An Aarray with more than two dimensions is called multi-dimensional array.

Ex:-
```
int [][][] matrix { {1,2,3}
                    {4,5,6}
                    {7,8,9}
                  };
system.out println (matrix[1][2][3]);
```

4) Jagged array

A multi-dimensional array where each row can have a different number of colours.

Ex:-
```
int[][] jagged = new int[3][];
jagged[0] = new int[] {1,2};
jadded[1] = new int[] {3,4,5};
jadded[2] = new int[] {6};
system.out.println (jagged[1][2]);
```